

PostgreSQL 数据库内核解读系列

第1节 PostgreSQL系统概述

概述，架构，编译安装，PG命令、调试

于巍（花名漠雪）

阿里云数据库开源社区Maintainer

目录

CONTENT

● 本系列教程介绍

● PostgreSQL 概述（历史、架构）

● PostgreSQL 安装启动

● PostgreSQL 常用命令、调试

1、本系列教程介绍

一：面向受众

PostgreSQL DBA、高校学生、内核开发爱好者

二：目标

- 1、PostgreSQL DBA遇到问题可以通过查看代码定位或解决简单问题，可以通过查看代码深入了解PG原理。
- 2、让没有接触过数据库内核的同学，了解内核基本原理、代码实现，可以完成简单功能开发；
- 3、共同学习、共同成长

三、课程策略

1. 章节设置参照彭智勇老师的《PostgreSQL数据库内核分析》，内容互补。
2. 每章节先介绍架构、原理、使用，再介绍代码。
3. 时间允许会结合之前合入Patch深入讲解，涉及PolarDB的修改也会提一些，或介绍相关前沿新技术。

第1节 PostgreSQL系统概述 - 概述，架构，编译安装，主要命令用法

第2节 PostgreSQL的体系结构 - 系统表，启动流程，查询流程，辅助进程

第3节 存储管理1 - 表和元组、磁盘管理、VFD、Buffer

第4节 存储管理2 - FSM、内存、Vacuum

第5节 索引 - 索引基本介绍、B-Tree索引、Hash索引、GiST索引、GIN索引、TSearch2全文搜索

第6节 查询编译 - 查询分析、查询重写、查询规划、代价估计、遗传算法

第7节 查询执行 - 查询执行策略、数据定义语句执行、可优化语句执行、计划节点、其他子功能介绍

第8节 事务处理与并发控制1 - 事务系统简介、事务保存点和子事务、两阶段提交、PostgreSQL的并发控制

第9节 事务处理与并发控制2 - PostgreSQL中的三种锁、锁管理机制、死锁处理机制、多版本并发控制

第10节 事务处理与并发控制3 - 日志管理、SLRU缓冲池、CLOG、SUBTRANS、MULTIXACT、XLOG

第11节 数据库安全 - 认证方法、基于角色的权限管理、对象访问控制

第12节 复制 - 流复制、三副本、逻辑复制

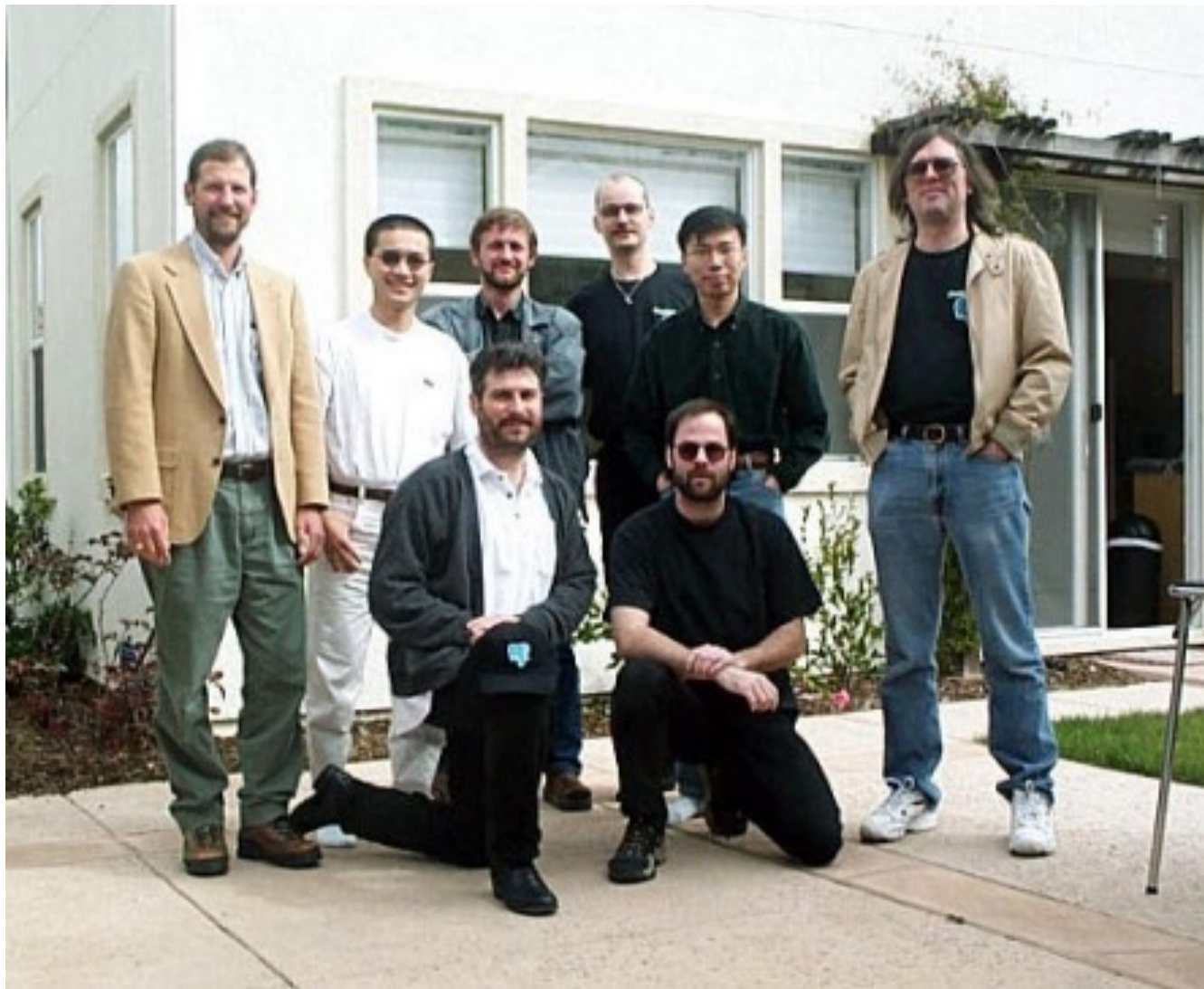
第13节 分布式1 - 分布式事务、分区管理、分布式算法

第14节 分布式2 - PGXC

第15节 分布式3 - Citus

2、PostgreSQL 概述（历史、架构）

PostgreSQL 历史



Top row: Thomas Lockhart, Jolly Chen, Vadim Mikheev Jan Wieck, Andrew Yu, Tom Lane

Bottom row: Bruce Momjian, Marc Fournier

1977 1985 1994 1996 1998 2016 2017 2018 2022

INGRES

- Berkeley POSTGRES项目
- 原型验证
- 1980 成立Ingres公司
- 商业版Illustra被 IBM informix收购

POSTGRES

Postgres95

引入SQL标准
Andrew Yu

PostgreSQL

开源

PG 6~7.1

MVCC
WAL
存储过程
外键
JOIN支持
临时表
TOAST

7.2~8.2

PITR
SCHEMA支持
Nonblocking Vacuum
GIN
Windows支持
ROLEs
dblink

PG 9 ~9.2

流复制

PG 11

分区表改进
存储过程及其事务
并行
JIT(just-in-time)编译

PG 15

PG11后每年一个版本
Zstd 压缩
逻辑复制增强



MICHAEL STONEBRAKER



JOLLY CHEN AND ANDREW YU

数据库领域开源生态

兼容现有生态的分布式数据库成为热点方向

关系型数据库

MySQL PostgreSQL

Scale up

Scale out

衍生场景

单机数据库增强

WebScaleSQL MariaDB
MyRocks Percona
openGauss **AliSQL**

Proxy/HA

Altas MGR
Orchestrator DB Proxy

Key-Value Cache/Store

Redis Codis Tendis
RocksDB LevelDB **Tair**
Cassandra ScyllaDB TiKV

阿里数据库已有开源产品

分库分表中间件

TDDL Cobar MyCat Zebra
Vitess Citus ShardingSphere

分布式数据库

TiDB CockroachDB
YugabyteDB OceanBase
PolarDB for PG
PolarDB-X

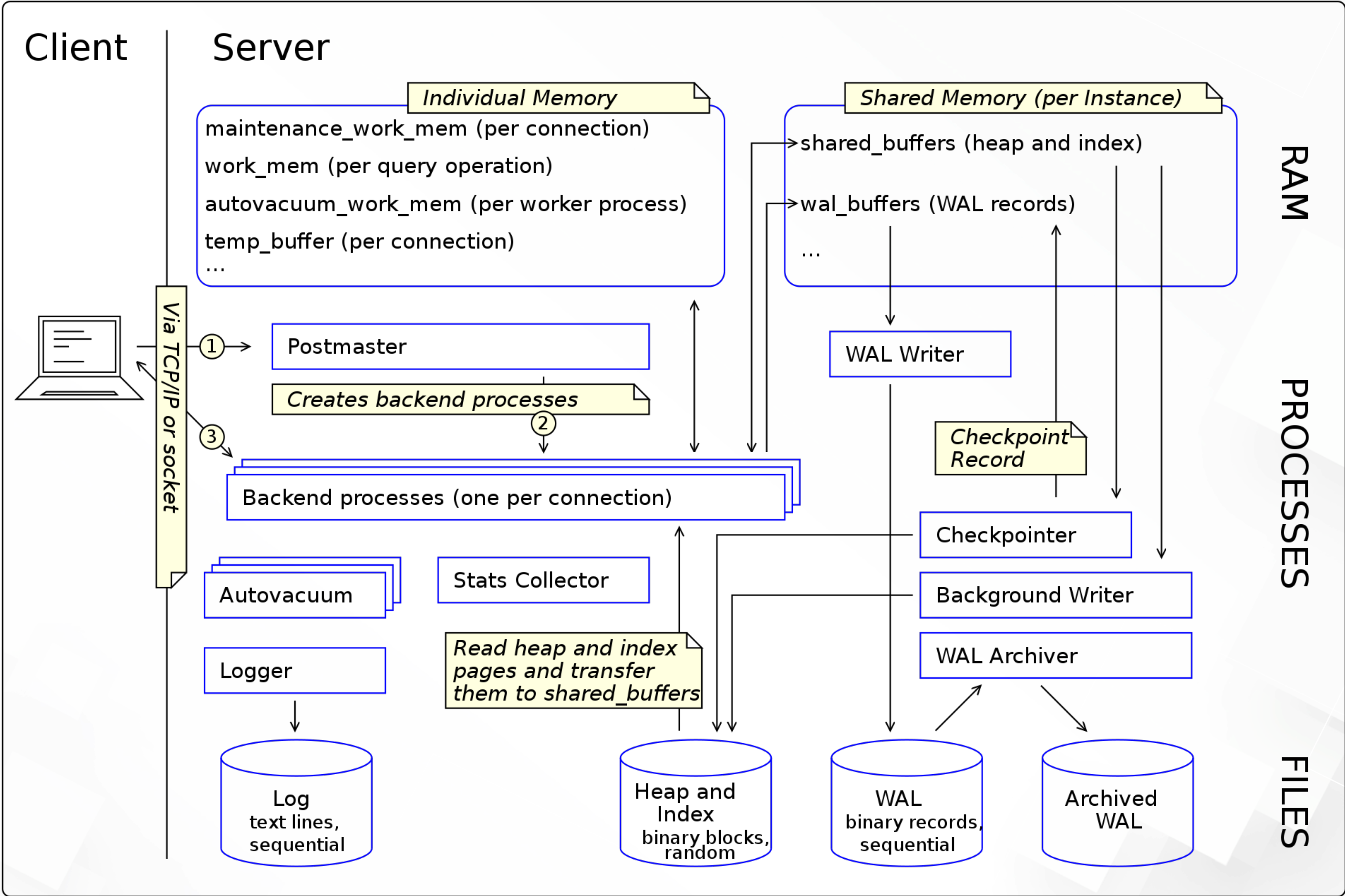
新数据模型

MongoDB CouchDB
Neo4j InfluxDB
ElasticSearch TDengine

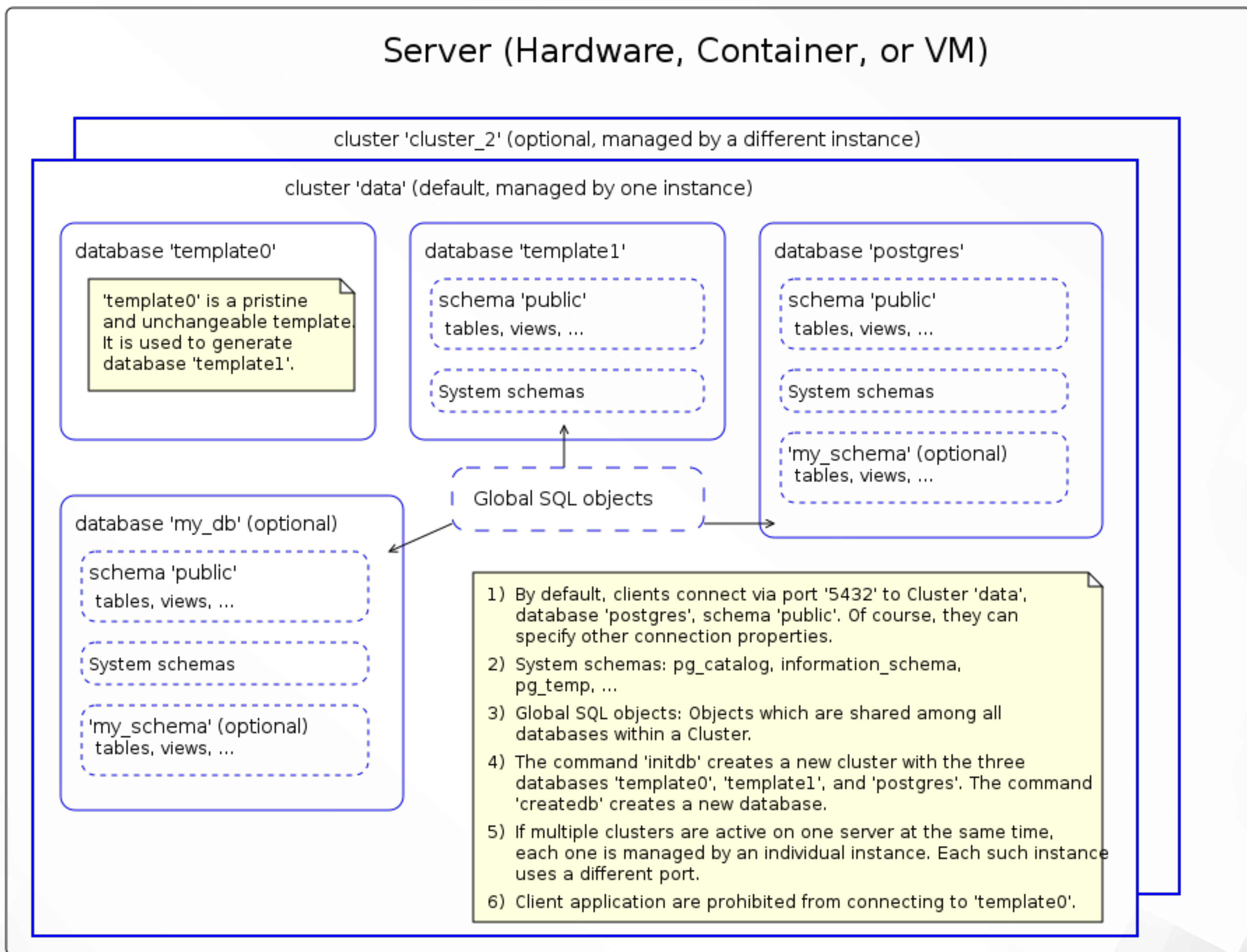
数据分析/流式计算

Hadoop Spark Presto
Flink Kafka Implu
Greenplum Clickhouse

PostgreSQL 架构



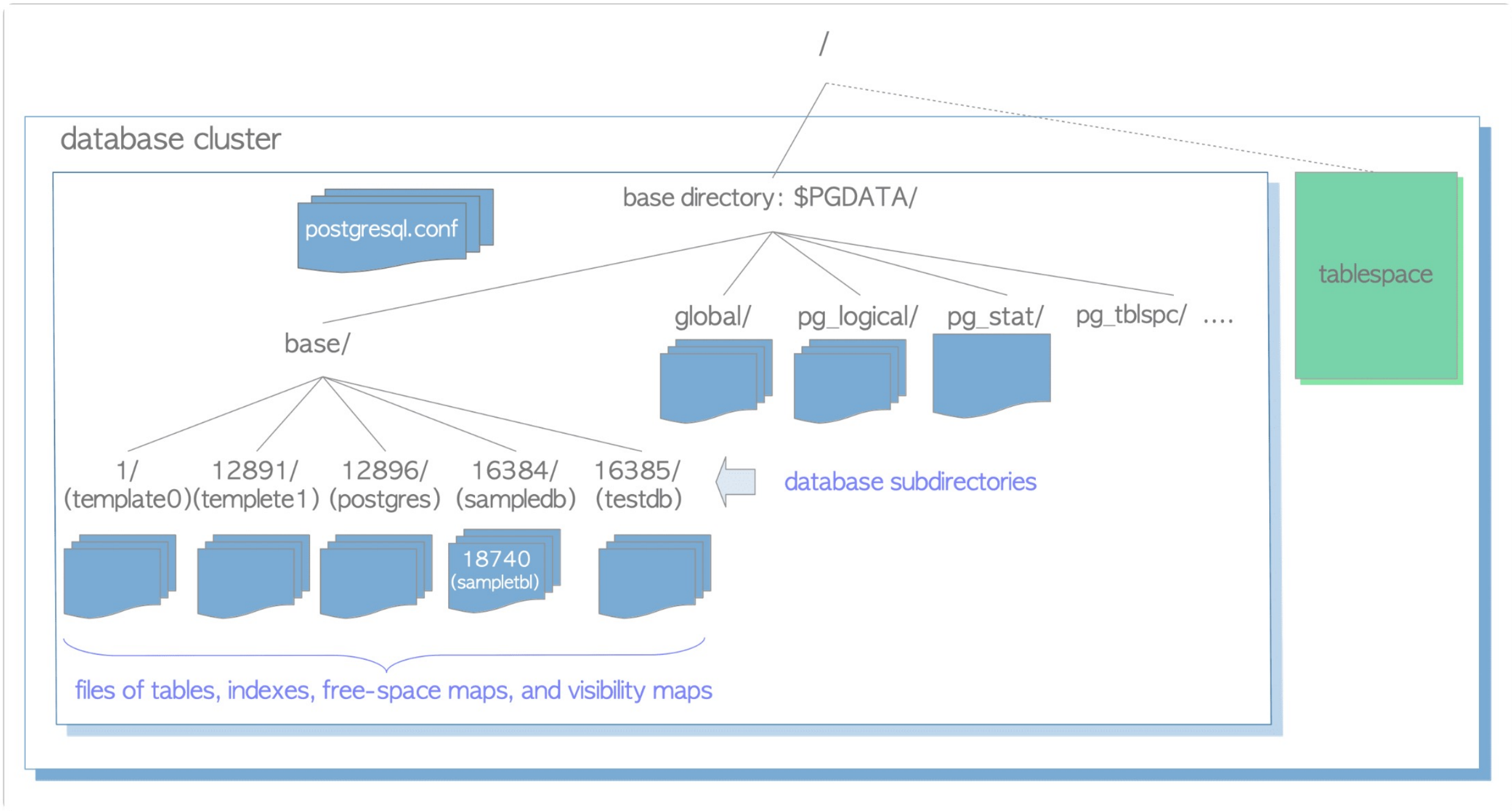
PostgreSQL 逻辑架构



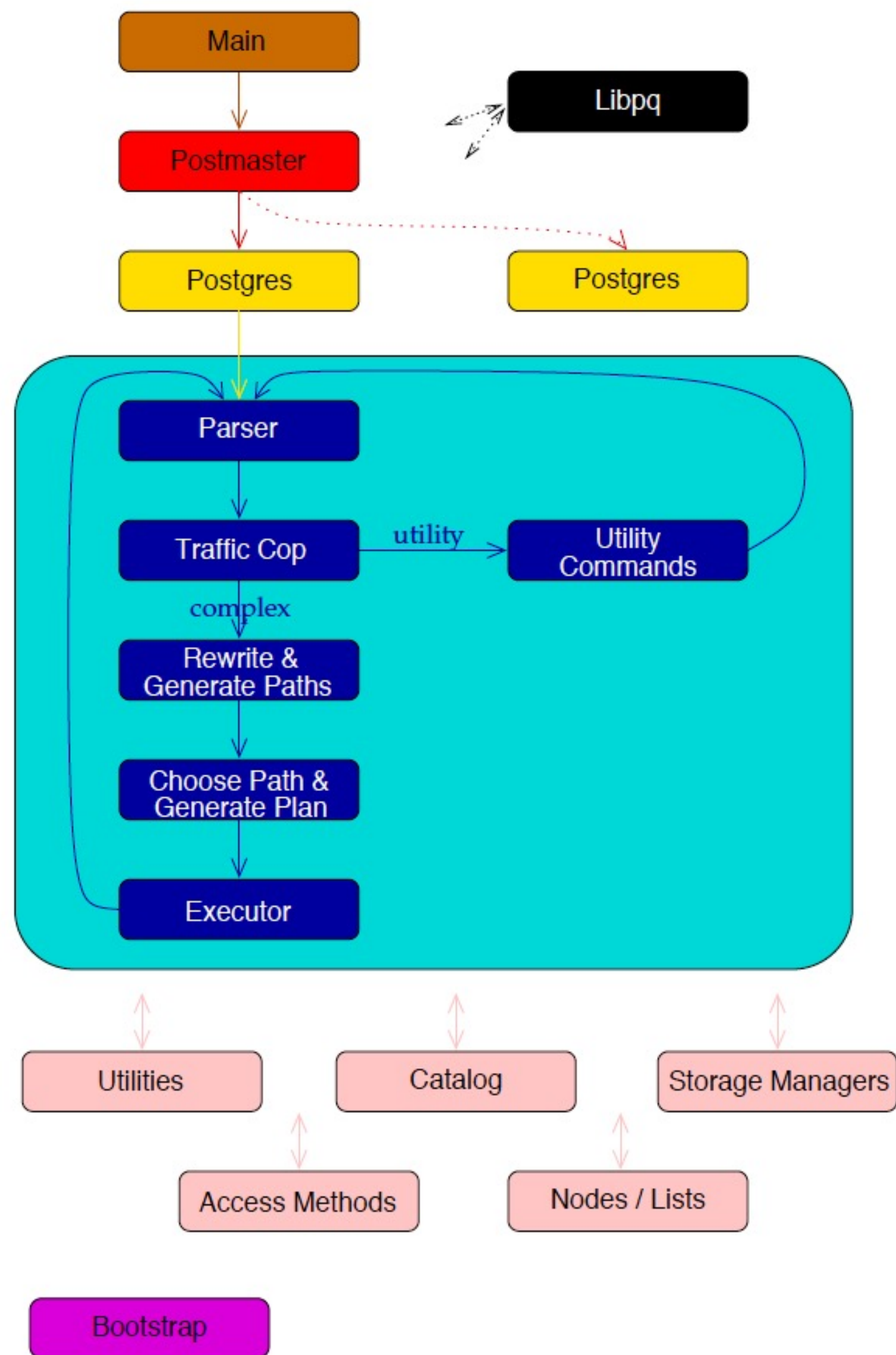
主要目录文件说明：

- base 每个数据库对于的子目录的子目录
- global 包含集簇范围的表的文件和全局控制信息等。
- pg_commit_ts 包含事务提交时间戳数据的子目录
- pg_dynshmem 包含被动态共享内存子系统所使用文件的子目录
- pg_logical 包含用于逻辑复制的状态数据的子目录
- pg_multixact 包含多事务状态数据的子目录（用户共享的行锁）
- pg_notify 包含LISTEN/NOTIFY状态数据的子目录
- pg_repslot 包含复制槽数据的子目录
- pg_serial 包含已提交的可序列化事务信息的子目录
- pg_snapshots 包含快照的子目录
- pg_stat 包含用于统计子系统的永久文件的子目录
- pg_stat_tmp 包含用于统计信息子系统临时文件的子目录
- pg_subtrans 包含子事务状态数据的子目录
- pg_tblspc 包含指向表空间的符号链接的子目录
- pg_twophase 用于预备事务状态文件的子目录
- pg_wal 保存预写日志
- pg_xact 包含事务提交状态数据的子目录, 记录事务提交状态数据

- postgresql.conf 参数文件
- pg_hba.conf 客户端认证控制文件(host-based access control)
- postgresql.auto.conf 参数文件，只保存alter system命令修改的参数
- postmaster.opts 记录服务器最后一次启动时使用的命令行参数
- postmaster.pid 记录进程的信息
- pg_version pg数据库主版本号文件



PostgreSQL 模块及代码目录结构



目录文件 说明

Makefile makefile

access/ 各种存储访问方法(在各个子目录下) **common**(共同函数)、**gin** (Generalized Inverted Index通用逆向索引)、**gist** (Generalized Search Tree通用索引)、**hash** (哈希索引)、**heap** (heap的访问方法)、**index** (通用索引函数)、**nbtree** (Btree函数)、**transam** (事务处理)

bootstrap/ 数据库的初始化处理(initdb的时候)

catalog/ 系统目录

commands/ SELECT/INSERT/UPDATE/DELETE以为的SQL文的处理

executor/ 执行器(访问的执行)

foreign/ FDW(Foreign Data Wrapper)

lib/ 公共函数

libpq/ 前端/后端通信处理

main/ postgres的主函数

nodes/ 构文树节点相关的处理函数

optimizer/ 优化器

parser/ SQL构文解析器

port/ 平台相关的代码

postmaster/ postmaster的主函数 (常驻postgres)

replication/ streaming replication

regex/ 正则处理

rewrite/ 规则及视图相关的重写处理

snowball/ 全文检索相关 (语干处理)

storage/ 共享内存、磁盘上的存储、缓存等全部一次/二次记录管理(以下的目录)**buffer/**(缓存管理)、**file/**(文件)、**freespace/**(Free Space Map管理) **ipc/**(进程间通信)、**large_object /**(大对象的访问函数)、**lmgr/**(锁管理)、**page/**(页面访问相关函数)、**smgr/**(存储管理器)

tcop/ postgres (数据库引擎的进程)的主要部分

tsearch/ 全文检索

utils/ 各种模块(以下目录) **adt/**(嵌入的数据类型)、**cache/**(缓存管理)、**error/**(错误处理)、**fmgr/**(函数管理)、**hash/**(hash函数)、**init/**(数据库初始化、postgres的初期处理)、**mb/**(多字节文字处理)、**misc/**(其他)、**mmgr/**(内存的管理函数)、**resowner/**(查询处理中的数据(buffer pin及表锁)的管理)、**sort/**(排序处理)、**time/**(事务的 MVCC 管理)

3、PostgreSQL 安装启动

PG 源码编译安装

命令演示

阿里云

1. 源码下载：

```
git clone https://github.com/postgres/postgres.git
cd postgres
git branch -r | grep -v '\->' | while read remote; do git branch --track "${remote#origin/}" "$remote"; done
git fetch --all
git pull --all
git branch
git checkout REL_11_STABLE
```

2. 源码编译安装：

```
./configure -help
./configure --prefix=/home/michael.yw/pghome --enable-debug --enable-cassert CFLAGS="-ggdb -O0 -g3 -Wall -fno-omit-frame-pointer -fstack-protector-strong" -- Debug版
./configure --prefix=/home/michael.yw/pghome --enable-debug --enable-cassert CFLAGS="-g -O2 -Wall -fno-omit-frame-pointer -fstack-protector-strong" -- Release版

make -sj && make install
```

3. 设置环境变量：

```
vi ~/.bashrc

export PATH="/home/michael.yw/pghome/bin:$PATH"
export LD_LIBRARY_PATH="/home/michael.yw/pghome/lib:$LD_LIBRARY_PATH"
export PGHOME="/home/michael.yw/pghome"
export PGDATA="/home/michael.yw/pgdata"

source ~/.bashrc
```

```
Optional Features:
--disable-option-checking ignore unrecognized --enable/--with options
--disable-FEATURE         do not include FEATURE (same as --enable-FEATURE=no)
--enable-FEATURE[=ARG]    include FEATURE [ARG=yes]
--disable-integer-datetimes
                           obsolete option, no longer supported
--enable-nls[=LANGUAGES]  enable Native Language Support
--disable-rpath            do not embed shared library search path in
                           executables
--disable-spinlocks       do not use spinlocks
--disable-atomics         do not use atomic operations
--disable-strong-random   do not use a strong random number source
--enable-debug            build with debugging symbols (-g)
--enable-profiling        build with profiling enabled
--enable-coverage         build with coverage testing instrumentation
--enable-dtrace           build with DTrace support
--enable-tap-tests        enable TAP tests (requires Perl and IPC::Run)
--enable-depend           turn on automatic dependency tracking
--enable-cassert          enable assertion checks (for debugging)
--disable-thread-safety   disable thread-safety in client libraries
--disable-largefile       omit support for large files
--disable-float4-byval    disable float4 passed by value
--disable-float8-byval    disable float8 passed by value
```


PG 初始化数据库启动

1. 初始化数据库

`initdb -D /home/michael.yw/pgdata`

```
$initdb
The files belonging to this database system will be owned by user "michael.yw".
This user must also own the server process.

The database cluster will be initialized with locale "en_US.UTF-8".
The default database encoding has accordingly been set to "UTF8".
The default text search configuration will be set to "english".

Data page checksums are disabled.

creating directory /home/michael.yw/pgdata ... ok
creating subdirectories ... ok
selecting default max_connections ... 100
selecting default shared_buffers ... 128MB
selecting default timezone ... Asia/Shanghai
selecting dynamic shared memory implementation ... posix
creating configuration files ... ok
running bootstrap script ... ok
performing post-bootstrap initialization ... ok
syncing data to disk ... ok

WARNING: enabling "trust" authentication for local connections
You can change this by editing pg_hba.conf or using the option -A, or
--auth-local and --auth-host, the next time you run initdb.

Success. You can now start the database server using:

    pg_ctl -D /home/michael.yw/pgdata -l logfile start
```

2. 参数修改

`vi /home/michael.yw/pgdata/postgresql.conf`

```
max_connections = 100
max_wal_senders = 10
max_replication_slots = 10
max_worker_processes = 10
shared_preload_libraries = 'pg_stat_statements'
listen_addresses = '*'
port = 6688
```

```
logging_collector = on
log_directory = 'log'
log_filename = 'postgresql-%a.log'
log_truncate_on_rotation = on
log_rotation_age = 1d
log_rotation_size = 0
log_checkpoints = on
log_connections = on
log_disconnections = on
log_error_verbosity = verbose
log_line_prefix = '%m [%p] '
log_timezone = 'PRC'
log_autovacuum_min_duration = 0
```

3. 数据库启动、停止：

`pg_ctl -D /home/michael.yw/pgdata -l /home/michael.yw/pgdata/logfile1 start`

`pg_ctl -D /home/michael.yw/pgdata -l /home/michael.yw/pgdata/logfile1 stop`

```
$ps -efl | grep michael
michael+ 37605      1  0 12:41 pts/20    00:00:00 /home/michael.yw/pghome/bin/postgres -D /home/michael.yw/pgdata
michael+ 37610    37605  0 12:41 ?          00:00:00 postgres: logger
michael+ 37612    37605  0 12:41 ?          00:00:00 postgres: checkpointer
michael+ 37613    37605  0 12:41 ?          00:00:00 postgres: background writer
michael+ 37614    37605  0 12:41 ?          00:00:00 postgres: walwriter
michael+ 37615    37605  0 12:41 ?          00:00:00 postgres: autovacuum launcher
michael+ 37616    37605  0 12:41 ?          00:00:00 postgres: stats collector
michael+ 37617    37605  0 12:41 ?          00:00:00 postgres: logical replication launcher
```


4、PostgreSQL 常用命令、调试

使用psql 连接

1. 连接数据库

```
psql -p 6688 -d postgres
```

2. 数据库基本操作

```
create table t1(a int, b text);
insert into t1 values(1, 'aa');
select * from t1;
```

3. psql 常用命令

```
psql -p 6688 -d postgres
```

连接数据库成功之后所有的命令都是使用 " \ " + 字符

- ? \ 所有命令帮助
- \l show databases , 列出所有数据库
- \di 列出连接数据库中所有index
- \dv 列出连接数据库中所有view
- \h sql命令帮助
- \dt show tables , 列出连接数据库中所有表
- \d
- \d后面什么都不带, 将列出当前数据库中的所有表。
- \d后面跟一个表名, 表示显示这个表的结构定义
- \q 退出连接
- \c [database_name] // 切换到指定数据库,相当于use
- \c show current DB name and user
- \conninfo 显示客户端的连接信息
- \du list all roles
- \du // 显示所有用户
- select version(); // 获取版本信息

Informational		
(options: S = show system objects, + = additional detail)		
\d[S+]		list tables, views, and sequences
\d[S+] NAME		describe table, view, sequence, or index
\da[S] [PATTERN]		list aggregates
\dA[+] [PATTERN]		list access methods
\db[+] [PATTERN]		list tablespaces
\dc[S+] [PATTERN]		list conversions
\dC[+] [PATTERN]		list casts
\dd[S] [PATTERN]		show object descriptions not displayed elsewhere
\dD[S+] [PATTERN]		list domains
\ddp [PATTERN]		list default privileges
\dE[S+] [PATTERN]		list foreign tables
\det[+] [PATTERN]		list foreign tables
\des[+] [PATTERN]		list foreign servers
\deu[+] [PATTERN]		list user mappings
\dew[+] [PATTERN]		list foreign-data wrappers
\df[anptw][S+] [PATRN]		list [only agg/normal/procedures/trigger/window] functions
\dF[+] [PATTERN]		list text search configurations
\dFd[+] [PATTERN]		list text search dictionaries
\dFp[+] [PATTERN]		list text search parsers
\dFt[+] [PATTERN]		list text search templates
\dg[S+] [PATTERN]		list roles
\di[S+] [PATTERN]		list indexes
\dl		list large objects, same as \lo_list
\dL[S+] [PATTERN]		list procedural languages
\dm[S+] [PATTERN]		list materialized views
\dn[S+] [PATTERN]		list schemas
\do[S+] [PATTERN]		list operators
\dO[S+] [PATTERN]		list collations
\dp [PATTERN]		list table, view, and sequence access privileges
\drds [PATRN1 [PATRN2]]		list per-database role settings
\dRp[+] [PATTERN]		list replication publications
\dRs[+] [PATTERN]		list replication subscriptions
\ds[S+] [PATTERN]		list sequences
\dt[S+] [PATTERN]		list tables
\dT[S+] [PATTERN]		list data types
\du[S+] [PATTERN]		list roles
\dv[S+] [PATTERN]		list views
\dx[+] [PATTERN]		list extensions
\dy[+] [PATTERN]		list event triggers
\l[+] [PATTERN]		list databases
\sf[+] FUNCNAME		show a function's definition
\sv[+] VIEWNAME		show a view's definition
\z [PATTERN]		same as \dp

1. 常用命令

```
cd /home/michael.yw/pghome/bin
ll /home/michael.yw/pghome/bin
createdb --help
createdb -p 6688 test;
pg_config
pg_ctl -help
pg_dump -help
pg_dump -p 6688 -d postgres -t t1 -f t1.sql;
cat t1.sql
pg_controldata
cd /home/michael.yw/pghome/lib/postgresql
```

2. 调试

```
ps -ef|grep michael
select pg_backend_pid();
pstack 37612
gdb postgres 96897
    bt -- 显示堆栈
    f 8 -- 查看堆栈中不同层级函数
    l -- 查看周围代码
    b exec_simple_query -- 设置断点
    info b -- 显示断点
    C+x a -- 查看代码
```

```
select * from t1;
```

```
[michael.yw@i22g03178.eu95sqa /home/michael.yw/pghome/bin]
$ll
total 42668
-rwxr-xr-x 1 michael.yw users 287192 Jun 6 10:18 clusterdb
-rwxr-xr-x 1 michael.yw users 280888 Jun 6 10:18 createdb
-rwxr-xr-x 1 michael.yw users 291984 Jun 6 10:18 createuser
-rwxr-xr-x 1 michael.yw users 265416 Jun 6 10:18 dropdb
-rwxr-xr-x 1 michael.yw users 265416 Jun 6 10:18 dropuser
-rwxr-xr-x 1 michael.yw users 1397824 Jun 6 10:18 ecpg
-rwxr-xr-x 1 michael.yw users 503352 Jun 6 10:18 initdb
-rwxr-xr-x 1 michael.yw users 214264 Jun 6 10:18 pg_archivecleanup
-rwxr-xr-x 1 michael.yw users 464272 Jun 6 10:18 pg_basebackup
-rwxr-xr-x 1 michael.yw users 496200 Jun 6 10:18 pgbench
-rwxr-xr-x 1 michael.yw users 203672 Jun 6 10:18 pg_config
-rwxr-xr-x 1 michael.yw users 306288 Jun 6 10:18 pg_controldata
-rwxr-xr-x 1 michael.yw users 297016 Jun 6 10:18 pg_ctl
-rwxr-xr-x 1 michael.yw users 1041856 Jun 6 10:18 pg_dump
-rwxr-xr-x 1 michael.yw users 322304 Jun 6 10:18 pg_dumpall
-rwxr-xr-x 1 michael.yw users 269616 Jun 6 10:18 pg_isready
-rwxr-xr-x 1 michael.yw users 358304 Jun 6 10:18 pg_receivewal
-rwxr-xr-x 1 michael.yw users 363120 Jun 6 10:18 pg_recvlogical
-rwxr-xr-x 1 michael.yw users 352800 Jun 6 10:18 pg_resetwal
-rwxr-xr-x 1 michael.yw users 594848 Jun 6 10:18 pg_restore
-rwxr-xr-x 1 michael.yw users 531584 Jun 6 10:18 pg_rewind
-rwxr-xr-x 1 michael.yw users 210792 Jun 6 10:18 pg_test_fsync
-rwxr-xr-x 1 michael.yw users 194240 Jun 6 10:18 pg_test_timing
-rwxr-xr-x 1 michael.yw users 532008 Jun 6 10:18 pg_upgrade
-rwxr-xr-x 1 michael.yw users 268720 Jun 6 10:18 pg_verify_checksums
-rwxr-xr-x 1 michael.yw users 760248 Jun 6 10:18 pg_waldump
-rwxr-xr-x 1 michael.yw users 30589488 Jun 6 10:18 postgres
lrwxrwxrwx 1 michael.yw users 8 Jun 6 10:18 postmaster -> postgres
-rwxr-xr-x 1 michael.yw users 1329752 Jun 6 10:18 psql
-rwxr-xr-x 1 michael.yw users 291952 Jun 6 10:18 reindexdb
-rw-r--r-- 1 michael.yw users 852 Jun 6 17:46 t1.sql
-rwxr-xr-x 1 michael.yw users 301992 Jun 6 10:18 vacuumdb
```

```
Breakpoint 1, exec_simple_query (query_string=0x27d4458 "select * from t1;") at postgres.c:915
915      {
(gdb) bt
#0  exec_simple_query (query_string=0x27d4458 "select * from t1;") at postgres.c:915
#1  0x00000000008f520d in PostgresMain (argc=1, argv=0x27ff538, dbname=0x27ff3b8 "postgres", user=0x27ff3b8) at postgres.c:100
#2  0x0000000000849cfd in BackendRun (port=0x27f7370) at postmaster.c:4422
#3  0x00000000008493ee in BackendStartup (port=0x27f7370) at postmaster.c:4086
#4  0x000000000084591b in ServerLoop () at postmaster.c:1728
#5  0x000000000084512e in PostmasterMain (argc=3, argv=0x27cef40) at postmaster.c:1401
#6  0x0000000000766a09 in main (argc=3, argv=0x27cef40) at main.c:228
```


开发工具-Source Insight

命令演示

阿里云

postmaster.c

Symbol Name (Alt+L)

StartWalWriter

StartWalReceiver

EXIT_STATUS_0

EXIT_STATUS_1

EXIT_STATUS_3

#if !defined WIN32

postmaster_alive_fds

#else

PostmasterHandle

#endif

PostmasterMain

CloseServerPorts

unlink_external_pid_file

getInstallationPaths

checkControlFile

DetermineSleepTime

ServerLoop

initMasks

ProcessStartupPacket

SendNegotiateProtocolVersion

processCancelRequest

canAcceptConnections

ConnCreate

ConnFree

ClosePostmasterPorts

reset_shared

SIGHUP_handler

pmdle

reaper

CleanupBackgroundWorker

CleanupBackend

HandleChildCrash

LogChildExit

PostmasterStateMachine

signal_child

SignalSomeChildren

TerminateChildren

BackendStartup

report_fork_failure_to_client

BackendInitialize

BackendRun

#if !defined EXEC_BACKEND

postmaster_forkexec

backend_forkexec

#if !defined WIN32

internal_forkexec

#else

internal_forkexec

#endif

SubPostmasterMain

#endif

ExitPostmaster

sigusr1_handler

process_startup_packet_die

process_startup_packet_quickdie

dummy_handler

HandleChildCrash(pid, exitstatus, _("startup process"));

continue;

}

/*

* Startup succeeded, commence normal operations

*/

StartupStatus = STARTUP_NOT_RUNNING;

FatalError = false;

Assert(AbortStartTime == 0);

ReachedNormalRunning = true;

pmState = PM_RUN;

connsAllowed = ALLOW_ALL_CONNS;

/*

* Crank up the background tasks, if we didn't do that already

* when we entered consistent recovery state. It doesn't matter

* if this fails, we'll just try again later.

*/

if (CheckpointnerPID == 0)

CheckpointnerPID = StartCheckpointner();

if (BgWriterPID == 0)

BgWriterPID = StartBackgroundWriter();

if (WalWriterPID == 0)

WalWriterPID = StartWalWriter();

/*

* Likewise, start other special children as needed. In a restart

* situation, some of them may be alive already.

*/

if (!IsBinaryUpgrade && AutoVacuumingActive() && AutoVacPID == 0)

AutoVacPID = StartAutoVacLauncher();

if (PgArchStartupAllowed() && PgArchPID == 0)

PgArchPID = pgarch_start();

if (PgStatPID == 0)

PgStatPID = pgstat_start();

/* workers may be scheduled to start now */

maybe_start_bgworkers();

/* at this point we are really open for business */

ereport(LOG,

(errmsg("database system is ready to accept connections")));

/* Report status */

AddToDataDirLockFile(LOCK_FILE_LINE_PM_STATUS, PM_STATUS_READY);

#if !defined USE_SYSTEMD

sd_notify(0, "READY=1");

#endif

StartCheckpointner

Macro in postmaster.c (src\backend\postmaster) at line 564

HANDLE childProcess, pid_t childPid);

#endif

static void ShmemBackendArrayAdd(Backend *bn);

static void ShmemBackendArrayRemove(Backend *bn);

#endif

/* EXEC_BACKEND */

#define StartupDataBase() StartChildProcess(StartupProcess)

#define StartBackgroundWriter() StartChildProcess(BgWriterProcess)

#define StartCheckpointner() StartChildProcess(CheckpointnerProcess)

#define StartWalWriter() StartChildProcess(WalWriterProcess)

#define StartWalReceiver() StartChildProcess(WalReceiverProcess)

/* Macros to check exit status of a child process */

#define EXIT_STATUS_0(st) ((st) == 0)

#define EXIT_STATUS_1(st) (WIFEXITED(st) && WEXITSTATUS(st) == 1)

#define EXIT_STATUS_3(st) (WIFEXITED(st) && WEXITSTATUS(st) == 3)

#if !defined WIN32

/*

* File descriptors for pipe used to monitor if postmaster is alive.

* First is POSTMASTER_FD_WATCH, second is POSTMASTER_FD_OWN.

*/

int postmaster_alive_fds[2] = {-1, -1};

#else

/* Process handle of postmaster used for the same purpose on Windows */

HANDLE PostmasterHandle;

#endif

/*

* Postmaster main entry point

*/

void

PostmasterMain(int argc, char *argv[])

{

int opt;

int status;

char *userOption = NULL;

bool listen_addr_saved = false;

Relation Referenced by Functions

Name

Lines of Code

File

Line

StartCheckpointner Callers

PostmasterStateMachine

reaper

PostmasterMain

ServerLoop

PostmasterMain

sigusr1_handler

153

196

477

142

477

99

postmaster.c (src\backend\postmaster)

postmaster.c (src\backend\postmaster)

postmaster.c (src\backend\postmaster)

postmaster.c (src\backend\postmaster)

postmaster.c (src\backend\postmaster)

postmaster.c (src\backend\postmaster)

564

3727

2885

679

1754

1401

5087

Project Files

Project Symbols

Folders

Symbol Categories

StartCheckpointner

Symbol

File Name

Metrics

StartCheckpointner

{anonCheckpointnerShmemStruct}::ckpt_started

postmaster.c (src\backend\postmaster)

checkpointner.c (src\backend\postmaster)

Line 2885 Col 35 reaper [UTF-8]

INS

